

9. Pastebin

Interviewer: Let's design Pastebin — a user pastes a chunk of text or code, we hand back a short link, and anyone with that link can view the paste. Some pastes should expire, some should be private. Straightforward, right? Walk me through it.

Candidate: It sounds simple, but the interesting tension is size: a paste can be one line or five megabytes of a log dump, and that size variance is what breaks a naive "just put it in a database row" design. Let me clarify the shape of the problem before I commit to anything.

1. Clarifying the requirements

Candidate: A few questions: 1. Max paste size — a few KB of code, or could someone paste a 50 MB dump? That changes where the content lives. 2. Expiry — time-based TTL, burn-after-read, or both? 3. Visibility — public (browsable), unlisted (link-only), or private (owner-auth)? 4. Are pastes ever edited, or immutable once written? 5. Read pattern — hit-once-and-forgotten, or do some go viral?

Interviewer: Cap it at a few MB — not a file-hosting service. Support both time-based TTL and burn-after-read. Public, unlisted, and private-with-owner should all exist. Pastes are immutable — create and delete only, no edits. Most pastes get a handful of reads, but a small fraction (a shared error message, a viral snippet) spike hard for a short burst.

Candidate: Good, that's a clean spec.

Functional - `POST /pastes {content, ttlSeconds?, burnAfterRead?, visibility?}` → returns a short key. - `GET /{key}` → returns the paste content. `DELETE /{key}` → owner-only. - Visibility: `public` / `unlisted` / `private`. Expiry: TTL and burn-after-read.

Non-functional — where the decisions live - **Payload size varies wildly** (100 bytes to several MB) — the body's home must not choke on the big end or waste money on the small end. - **Read-heavy, but unevenly** — most pastes get a handful of reads; a minority spike hard. - **Cheap storage at scale** — millions of pastes/day, most nobody looks at after a day. - **Correctness on burn-after-read** — exactly one reader must succeed; the sharp concurrency edge of this problem. - **Read latency < 200 ms**, nice-to-have, not a brutal checkout-style SLA.

2. Estimation

5M new pastes/day, avg 50 KB (mix of one-liners and multi-MB dumps),
avg 5 reads per paste over its lifetime.

writes/sec = $5,000,000 / 86,400 \approx 58/\text{sec}$ (peak $\sim 300/\text{sec}$)
reads/sec = $5 \times \text{writes} \approx 290/\text{sec}$ (peak $\sim 1,500/\text{sec}$ on a viral spike)

Blob storage (the big number):

5M/day * 50KB = 250 GB/day raw text $\rightarrow \sim 90$ TB/yr if nothing expired
 $\rightarrow$ most pastes carry a short TTL, so steady-state after GC is tens of TB.

Metadata storage (the small number):

~ 200 bytes/row (key, owner, ttl, visibility, blob pointer, size)
5M/day * 200B * 365 ≈ 36 GB/yr of rows $\rightarrow$ trivial for any DB.

Key space: base62, 7 chars $\rightarrow 62^7 \approx 3.5$ trillion. Not a constraint.

$\Rightarrow$ Metadata is small and cheap no matter what we pick; the BLOB needs
a storage tier built for large, immutable objects. That's the fork.

3. The naive V1 — one table, and why the blob breaks it

Interviewer: Start simple. What's the most obvious thing that could work?

Candidate: One table, blob and metadata in the same row:

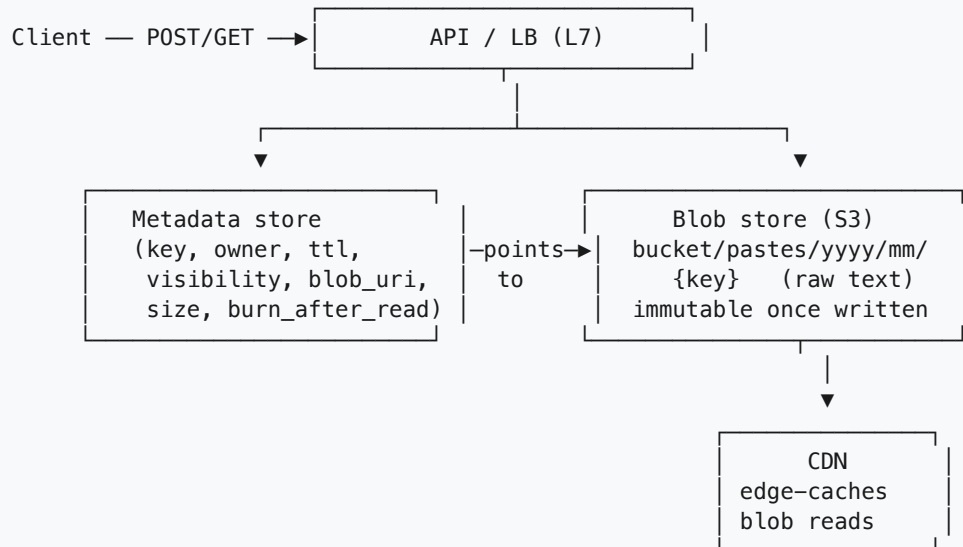
```
[Client] --POST /pastes--> [App Server] --> [Postgres: pastes(key, content, ttl, ...)]  
[Client] --GET /{key}----> [App Server] --> [SELECT content FROM pastes WHERE key=?]
```

Why it breaks: - **Row/page bloat** — a 5 MB `content` column shares pages with thousands of 200-byte rows; every backup/replica sync moves blob weight it doesn't need. - **Cost** — provisioned SSD-backed DB storage is far pricier per GB than object storage. - **No CDN-ability** — a DB row can't be edge-cached; an object-store URL can. - **Buffer-pool pollution** — a few fat blobs evict useful small rows from DB cache.

Candidate: So V1 works at toy scale, but the fix is obvious: **split the blob out of the database.**

4. The key idea — metadata in a DB, blob in object storage

Candidate: Metadata (key $\rightarrow$ owner, ttl, visibility, blob pointer, size) is a tiny, single-key lookup — a KV store or plain SQL table is fine either way at this volume. The **paste body** is large, immutable once written, read wholesale — exactly the shape object storage (S3-class) is built for: cheap per-GB, hugely durable, natively CDN-cacheable.



Write order matters: blob **first**, metadata **second**. If metadata pointed at a blob before it existed, a reader could hit a dangling pointer. Blob-first means the worst failure is a harmless orphaned blob, easy to GC.

5. Key generation

Interviewer: How do you generate the short key, and what if two writers collide?

Candidate: Random base62 (7 chars), attempted via a **conditional insert** (`INSERT ... ON CONFLICT DO NOTHING` , or `attribute_not_exists` in a KV store); on collision, regenerate and retry. At 3.5 trillion combinations vs. millions of keys, collisions are rare enough that this beats running a separate pre-allocated key-range service — I'd only reach for that added machinery if key generation itself became a bottleneck, which it won't here.

6. Scaling reads — 10M pastes/day and a viral spike

Interviewer: Scale it up — 10M pastes/day, and every so often one paste gets hammered by thousands of readers in an hour. Walk me through the read path.

Candidate: Reads split into two lookups — metadata, then blob — and both get short-circuited by caching before touching origin.

READ FLOW – GET /{key}

Client



[CDN / Edge] — cache key = paste key; HIT returns raw content, nothing else touched
| MISS



[App Server]

└ Redis: GET meta:{key}

 HIT → use cached {blob_uri, visibility, burn_after_read, expires_at}

 MISS → read metadata store → populate Redis (TTL ≤ paste's expires_at)

└ if burn_after_read: atomic conditional delete-on-read (§11) – special path

└ fetch blob from S3 (or CDN already served it) → return content

- **CDN absorbs the viral spike almost entirely** — immutable content behind a stable URL, the textbook CDN case. Thousands of reads/hour become a handful of origin fetches.
- **Redis in front of metadata** catches "valid key / burn-after-read / expired" without hitting the DB — useful for the long tail of pastes read a few times, scattered enough that the CDN object may already have been evicted between reads.
- **Non-viral pastes barely need caching, and that's fine** — a single-key lookup + one blob GET is cheap on its own; caching exists for the tail, not the median.

7. Scaling writes

Interviewer: Now the write side at 10M/day — what happens on POST /pastes ?

WRITE FLOW – POST /pastes

Client —(chunked upload)→ [App Server]

1. generate candidate key

2. PUT blob to S3: bucket/pastes/{yyyy}/{mm}/{key} (fails? abort, nothing else happened)

3. conditional INSERT metadata row (key taken? regenerate, retry step 3)

4. return {key, url, expiresAt?}

- **Stream/chunk the upload** rather than buffering in app-server memory.
- **Enforce max size at the LB/gateway**, before it reaches an app server or S3.
- **Idempotency matters more than a URL shortener** — bigger payloads, more retried timeouts. Accept a client Idempotency-Key ; cache key → {pasteKey, url} ~24h so a retry returns the original result instead of creating a duplicate live link.

8. Multi-region + consistency

Interviewer: Go global — US, EU, Asia all creating and reading pastes. What replicates, and what's the consistency story?

- **Blob store:** S3 already replicates multi-AZ for durability. For cross-region *latency*, I'd lean on **CDN edge PoPs** caching reads close to users with a single-region origin, rather than replicating every

blob everywhere — blobs are immutable, so edge caching alone solves most of the latency problem.

- **Metadata store:** replicate across regions (leader-follower/quorum). Metadata is write-once, so **replica lag is harmless** for normal reads — an eventually-consistent read moments after creation isn't a correctness problem.
- **The one place staleness bites: burn-after-read.** If replication lag hides a delete, two regions could both serve a "read once" paste. That delete must go through the metadata store's **strongly consistent leader/quorum path**, pinned to the home region, even though everything else about metadata is eventually consistent globally.

9. Latency — where the milliseconds go

Candidate: For a Singapore reader of a paste created in us-east-1:

Singapore client -> nearest CDN edge PoP	~5–20 ms	(edge is close)
CDN MISS -> edge fetches origin S3 (us-east-1)	~150–200 ms	(cross-Pacific)
metadata lookup (Redis, regional)	~1–5 ms	(if region-local)
metadata MISS -> metadata DB read	~10–30 ms	

Two levers: **CDN hit rate dominates** — once warm at the Singapore edge, reads are 5-20 ms; the fix isn't "faster S3," it's keeping popular content warm at the edge. And **keep metadata regional too** — if metadata only lives in us-east-1, every read pays a cross-region round trip just to resolve `key` → `blob_uri` even after the blob is cached, defeating half the CDN's point.

10. Async — cleanup and analytics off the critical path

Interviewer: Pastes expire. How does deletion actually happen, and what shouldn't be on the request path?

```
[ Metadata store: expires_at, indexed ]
  ▼
[ Sweeper job, every N min ] SELECT key, blob_uri WHERE expires_at < now() LIMIT 1000
  ▼
[ Delete metadata row ] → [ Enqueue blob delete ] → [ S3 DeleteObject ]
                               (or: S3 lifecycle rule on the {yyyy}/{mm}/ prefix,
                               letting S3 expire objects natively — a backstop)
```

- **Metadata deletion is the source of truth** for "does this paste exist" — the S3 delete can lag behind, harmlessly, since nobody can reach it via a deleted metadata row anyway.
 - **Read-count/analytics** are emitted as events to a stream and aggregated async — never a synchronous counter write per read.
 - **Burn-after-read's delete is the one exception — it's NOT async.** It must be synchronous and atomic on the read path itself, because correctness depends on it happening before the response is served.
-

11. Concurrency — the burn-after-read race

Interviewer: Two people click the same burn-after-read link at nearly the same instant. What happens?

Candidate: This is the one spot where "eventually consistent" fails us — exactly one caller must get the content, enforced by the datastore, not application code (two racing app servers will occasionally both "win").

```
SQL: DELETE FROM paste_metadata WHERE paste_key = ? AND burn_after_read = true
      RETURNING blob_uri;
      -- exactly one caller's DELETE returns a row; the other gets zero rows

KV:   DeleteItem(key) WHERE ConditionExpression "attribute_exists(paste_key)"
      -- exactly one caller succeeds; the other gets a ConditionalCheckFailed
```

Winner fetches the blob and returns it (blob delete happens async after — no rush, metadata is already gone). Loser gets a 404, indistinguishable from the key never existing. This must run against the strongly consistent leader/quorum path (§8) — a stale replica could let both requests see "unread" and both succeed.

12. Caching — the full picture

Pastes are **immutable after creation**, so invalidation is nearly a non-problem: an entry is either "still valid" or "the paste's own TTL passed" — no third case of content changing underneath the cache.

Layer	What's cached	Invalidation
CDN edge	Raw blob content, keyed by paste key	TTL capped at <code>expires_at</code> ; burn-after-read responses marked no-store , never cached
Redis (metadata)	key -> {blob_uri, visibility, ttl, burn_after_read}	TTL bounded by the paste's expiry; proactive DEL on a successful burn, on top of the source row already being gone
Blob store (S3)	N/A — origin, not a cache	Lifecycle rules remove objects past their prefix's age

The one deliberate exception: **burn-after-read content is never cached anywhere** — caching directly conflicts with "readable exactly once." Everything else is cached aggressively because it's immutable.

13. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Paste body (blob)	Object storage (S3-class)	Cheap per-GB at tens-of-TB scale, 11-nines durability, natively servable behind a CDN. <i>Not a DB column</i> — multi-MB rows bloat

Concern	Choice	Why this, and why not the alternative
		pages/buffer-pool, cost more on provisioned SSD storage, and can't be edge-cached the way an object URL can.
Paste metadata	KV store or plain SQL (either works)	Single-key lookup, no joins, small fixed-shape rows. <i>Not storing metadata in S3 object tags</i> — that forces a blob-store round trip just to check visibility/TTL before you know whether to serve anything.
Read cache (metadata)	Redis, TTL $\leq$ paste's expiry	Sub-ms validity checks, keeps the metadata DB off the hot path. <i>Not skipping it</i> — fine on average, but wasteful under a viral spike's repeated key checks.
Read cache (blob)	CDN in front of the object store	Absorbs viral-spike reads at the edge — same trick that makes hot URL-shortener redirects cheap. <i>Not serving blobs through app servers every time</i> — wastes compute on cache-hit-able requests.
Burn-after-read	Atomic conditional delete-on-read, strongly consistent	Guarantees exactly one winner. <i>Not "check flag, then separate DELETE"</i> — a read-then-write race lets two concurrent readers both pass the check.
Key generation	Random base62 + conditional insert, retry on collision	Simple, no extra service; collision rate negligible at 3.5T keys. <i>Not pre-allocated ranges</i> — needless machinery at a few hundred writes/sec.
Expiry cleanup	Sweeper on <code>expires_at</code> + S3 lifecycle rules as backstop	Keeps deletion off the request path; two independent layers mean one bug doesn't leave orphans forever. <i>Not sync delete-checks on every read</i> — adds latency for a background concern.
Read analytics	Async event stream, aggregated off-path	A slow analytics pipeline must never slow a live read. <i>Not a sync counter increment per read</i> — needless write contention on a hot key.

Say-it-out-loud justification: "Metadata is small and cheap no matter what store I pick, so it lives in a KV/SQL table for fast single-key lookups; the paste body is large, immutable, and read wholesale, so it belongs in object storage behind a CDN — never in a database row. Writes go blob-first, metadata-second, so metadata never points at content that doesn't exist. Everything is cached aggressively because pastes never change after creation — except burn-after-read, which bypasses every cache and resolves through a strongly consistent atomic delete-on-read, because 'readable exactly once' is a correctness guarantee, not a performance knob."

14. Failure modes & graceful degradation

Failure	What happens	Mitigation
Blob write ok, metadata write fails	Orphaned blob, paste never "exists" to users	Harmless by write order; GC sweep clears blobs with no metadata row
Metadata write ok, blob write fails	Would be a dangling pointer — must never happen	Prevented structurally by blob-first, metadata-second

Failure	What happens	Mitigation
S3 down	Creates fail fast	Don't accept metadata for content never stored; cached reads keep serving for a while
Metadata store node down	Degraded reads/writes	Replicas/quorum keep serving; burn-after-read fails closed rather than risk a double-read
CDN/cache down	Slower, not wrong	Fall back to direct blob-store reads
Missing/expired key	404	Never 500; don't leak whether an unlisted/private key ever existed
Sweeper falls behind	Expired pastes linger briefly	S3 lifecycle rules are an independent backstop

15. Follow-up curveballs

Interviewer: How do you stop someone scripting thousands of paste creations a minute — spam, malware hosting, scraping?

Candidate: Rate-limit `POST /pastes` per IP/account at the gateway (writes are far more expensive than reads, so throttle them hardest). CAPTCHA for anonymous creation past a low threshold. Async content scanning on new blobs, off the critical path, with the ability to retroactively pull a flagged paste.

Interviewer: Someone wants to support editing a paste — same link, new content.

Candidate: Breaks the immutable-blob assumption my caching relies on. I'd model it as **versioning, not mutation**: each edit writes a new blob (`.../{key}/v2` , `v3` , ...), and metadata's `blob_uri / current_version` points at the latest. Key CDN cache entries by `{key, version}` so an edit naturally busts the cache by pointing readers at a new key, instead of requiring a push-based invalidation.

Interviewer: A paste goes viral — 50,000 reads in ten minutes. What's the actual bottleneck?

Candidate: Cold cache-miss traffic converging on one S3 object before every edge PoP has warmed — a thundering herd of near-simultaneous misses. Mitigation: **request coalescing** at the CDN (collapse concurrent misses for the same key into one origin fetch) plus an **origin-shield** layer so multiple PoPs missing at once still only generate one upstream GET. CDN origin-fetch rate, not app-server CPU, is the signal that tells you a paste went viral.

Interviewer: What if many users paste the exact same content — five copies of the same log dump?

Candidate: Hash content (`SHA-256`) at write time and store at a **content-addressed path** (`bucket/pastes/{hash}`); multiple metadata rows can point at the same blob. The cost: deletion

becomes **reference-counted** — you can't delete the blob until every referencing row has expired. Real complexity, so I'd only add it if duplicate-content volume justified it; one blob per paste is the simpler, correct default.

16. Deep-dive: "Why not just put the paste body in the database?"

Interviewer: You keep splitting the blob into object storage. Simplest possible counter-argument: Postgres supports a `TEXT` column up to 1 GB, or a `BYTEA / BLOB`. Why not just store the paste content in the same row as the metadata and skip S3 entirely? One table, one write, one read, no second system to operate.

Candidate: It's the right question, because at small scale it genuinely *is* fine — I want to be precise about where it stops being fine, not just say "never do this."

What actually breaks, mechanically

- **Row/page bloat compounds with every backup and replica.** A 3 MB `content` column doesn't just sit there — every `pg_dump`, every WAL-shipped replica sync, every streaming replication byte carries that blob weight along with it, even though only a fraction of pastes are ever read again. Metadata-only backups would be trivial; blob-laden backups scale with total bytes ever written, not with what's actually being served.
- **Buffer-pool pollution.** Postgres's shared buffer cache is sized for the *working set*. A handful of multi-MB rows scanned or vacuumed can evict thousands of small, hot metadata rows from cache, turning fast single-key lookups into disk reads — collateral damage to unrelated traffic.
- **VACUUM/replication cost scales with blob churn, not row count.** Postgres MVCC keeps old tuple versions around until vacuumed; large TOAST'd columns make autovacuum and WAL replay measurably heavier per write, even though the paste is immutable and never updated.
- **Read throughput ceiling.** A DB connection pool is a scarce, expensive resource sized for fast metadata-shaped queries. Streaming a 5 MB blob through that same connection pool holds the connection (and a backend process) for the full transfer time — one viral paste can pin a meaningful slice of your pool.
- **No CDN-ability — this is the one that doesn't show up in a load test.** An object-store URL (`https://cdn.example.com/pastes/{key}`) can be edge-cached and served without ever touching origin again. A row behind `SELECT ... WHERE key = ?` cannot be put behind a CDN at all — every single read, forever, round-trips to your database. For the viral-spike case (§6), this alone is decisive: object storage turns "10,000 reads/hour" into "one origin fetch, 9,999 cache hits"; blob-in-DB turns it into 10,000 database reads.

Comparison table

Concern	Blob in DB (<code>TEXT</code> / <code>BYTEA</code> column)	Object store + metadata DB (chosen)
Cost per GB	Provisioned SSD-backed DB storage — several times pricier per GB	Object storage tiers — cents per GB, built for exactly this shape

Concern	Blob in DB (<code>TEXT</code> / <code>BYTEA</code> column)	Object store + metadata DB (chosen)
Backup/replica cost	Every backup and replica carries full blob weight, growing forever	Metadata backups stay tiny; blobs replicate independently (S3 already multi-AZ)
Buffer-pool / cache impact	Large rows evict small hot rows from shared cache	DB cache stays 100% metadata — small, hot, cheap to keep warm
CDN-ability	None — every read hits the DB, no edge caching possible	Native — immutable object behind a stable URL, edge-cacheable
Read throughput under a spike	Every read holds a DB connection for the full transfer	CDN absorbs the spike; origin barely notices
Write path complexity	One write, one system, transactionally simple	Two writes (blob then metadata), needs the blob-first ordering rule (§4)
Operational surface	One system to run	Two systems (DB + object store) — more moving parts
Best at	Tiny payloads, low read volume, want transactional simplicity	Large and/or highly-read payloads, want CDN offload and cheap storage at scale

The threshold — where inlining is actually the right call

Candidate: I wouldn't dogmatically split every paste. If the vast majority of pastes are small — say **under 1-2 KB**, a one-liner or a short snippet — the *entire* content fits comfortably inside a single metadata row alongside its own index entry, with none of the bloat/vacuum/cache costs above, because the row never gets big enough to TOAST or dominate a page. Many real systems use a **hybrid**: inline content under a threshold directly in the metadata row (`inline_content` column, nullable); above the threshold, write to object storage and store `blob_uri` instead. That collapses "create + fetch" into one round trip for the common small-paste case, while keeping the large/viral tail off the database entirely — the two systems' costs only show up when they're actually justified.

Rule of thumb: *If the payload is small enough that a thousand of them together don't stress your buffer pool, and you don't need CDN-level read fan-out, inlining in the DB is simpler and correct. Once size or read-fan-out enters the picture — which a paste's "0 to viral" shape guarantees for a meaningful minority — object storage's cost-per-GB and CDN-ability win outright.*

17. Deep-dive: expiry and cleanup at scale — the race at the exact moment a paste dies

Interviewer: Zoom into cleanup. You've got millions of pastes with TTLs, all expiring on their own schedule, plus burn-after-read pastes that should die on their *first* read. Walk me through what can go wrong at the exact boundary — someone reading a paste at the instant it expires, or two people reading a burn-after-read paste at the same instant.

Candidate: There are really two different races hiding here, and they need different answers. Let me take them one at a time.

Race 1 — reading exactly at TTL expiry

A TTL-based paste doesn't have a single reader to serialize against — it just has a clock. The danger isn't "two readers fight over it," it's **inconsistency between where expiry is enforced and where the data actually lives**:

```
t = expires_at - 1ms: GET /{key} arrives, reads Redis metadata cache (not yet evicted)
                      -> looks valid, proceeds to fetch blob from S3
t = expires_at:       TTL fires in Redis; sweeper hasn't run yet; S3 object still exists
t = expires_at + 1ms: blob fetch completes, request succeeds
```

Was that correct? Debatable — the paste "expired" mid-request. Any answer between "still valid" and "already gone" is defensible AS LONG AS every layer agrees on a single source of truth for "is it expired" at request time.

The actual danger is when caches disagree with each other, not when a request straddles the boundary by a few milliseconds:

BAD: CDN edge cache TTL > paste's expires_at
-> paste is logically expired everywhere else, but the CDN keeps serving the cached blob for however much longer its own TTL says, from a different clock than the metadata store's.

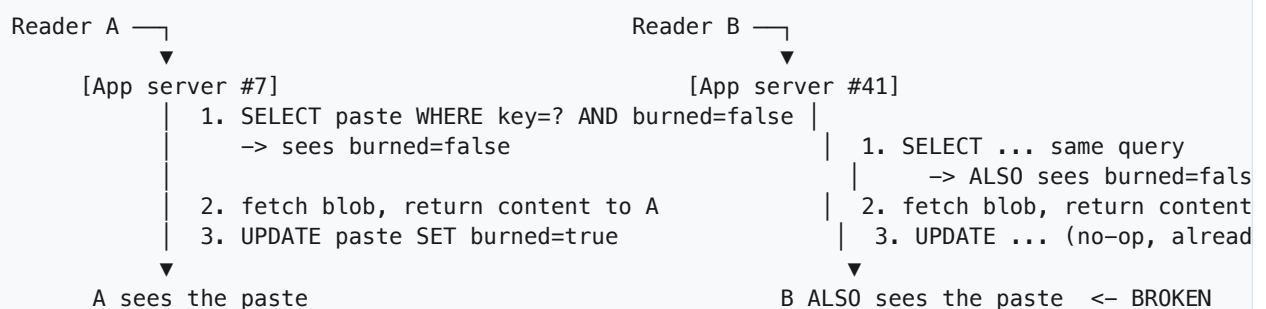
GOOD (\$9's rule, restated): every cache layer's TTL is capped AT OR BELOW the paste's own expires_at. A request that lands 1ms before expiry reading slightly stale-but-still-valid data is fine; a request landing 1 HOUR after expiry still getting served by a cache that doesn't know the paste died is not.

So Race 1's fix isn't a lock — it's **discipline**: bound every cache TTL by expires_at, and accept that "expired" has a few-millisecond fuzz at the exact boundary, same as any TTL-based system (browser caches, DNS, session cookies). Nobody needs a linearizable clock for this.

Race 2 — burn-after-read under concurrent readers

This one is a real correctness race, not fuzz — because there's a person, not a clock, deciding who wins, and the promise is "exactly one reader, ever."

Two readers click the same burn-after-read link within the same millisecond:



Why this happens: "check, then act" is two separate operations with a gap between them. Both app servers ran step 1 before either ran step 3 — a classic **read-then-write TOCTOU race**. It doesn't matter

that they're different processes on different servers; the database never enforced mutual exclusion because the check and the mutation weren't one atomic unit.

Ranked fixes

1. **Atomic delete-on-read / conditional mutation — the fix, not a mitigation.** Collapse "check" and "act" into a single atomic operation the datastore itself serializes: `` SQL: DELETE FROM paste_metadata WHERE paste_key = ? AND burn_after_read = true RETURNING blob_uri; -- the DB guarantees only one concurrent DELETE can match+return a row; -- the loser's DELETE affects zero rows, full stop.

Redis: GETDEL key -- atomic get-and-delete, single round trip -- or, if you need a condition beyond existence: WATCH/MULTI, or a Lua script (GET; if exists DEL and return value) so check+delete is one atomic script execution, no gap. `` This is what §11 already prescribes for this exact scenario – worth restating here because it's the one lock that actually must be real, not eventual. 2. **Lazy expiry (check-on-access) for TTL pastes – fine, and the default.** Don't proactively hunt for TTL'd pastes on every read; just compare `expires_at < now()` when a read happens, and treat expired-but-not-yet-swept rows as 404. Cheap, correct, no background work required for correctness (the sweeper is about **reclaiming storage**, not about **correctness** – an unswept expired row is already unreachable because the read path checks the timestamp). 3. **Active expiry (sweeper / TTL index) – needed for storage reclamation, not correctness.** §10's sweeper (or a Redis key TTL, or a Mongo TTL index) actually frees space. Skipping it doesn't cause wrong answers, it causes an ever-growing table of dead rows. Run it, but don't confuse it with the burn-after-read guarantee above – a sweeper running every 5 minutes does nothing for a race that happens in the same millisecond. 4. **TTL on the object store vs. the DB – they answer different questions.** S3 lifecycle rules expire the **blob** (reclaim storage cost); the metadata row's `expires_at/delete` is what makes the paste *unreachable*. Never let the object-store TTL be the enforcement point for "can this be read" – S3 lifecycle deletions run on their own schedule (up to ~24-48h lag in practice) and a paste must stop being servable the instant its logical expiry or its one-time-read fires, not whenever S3 gets around to garbage-collecting the object. Metadata deletion is the source of truth (§10); object-store TTL is a cost backstop.

Recommended approach

Candidate: Lazy timestamp-check for ordinary TTL expiry — it's correct with zero extra infrastructure, since "expired" is just a comparison against a value already in the row. An active sweeper plus S3 lifecycle rules run underneath purely to reclaim storage, on their own schedule, never gating correctness. Burn-after-read is the one case that needs a *real* atomic primitive — DELETE ... RETURNING or Redis GETDEL /a Lua script — because two humans racing for the same one-time link is a correctness problem, not a storage-cost problem, and only the datastore's own atomicity can guarantee exactly one winner.

What to say out loud: "TTL expiry and burn-after-read look like the same problem — 'something disappearing' — but they need different tools. Ordinary expiry is a timestamp comparison with lazy checks on read and a sweeper for storage cleanup; a request landing near the boundary is fuzzy by nature and that's fine, the same way a browser cache TTL is fuzzy. Burn-after-read is different: two readers racing for one link is a TOCTOU bug if you 'check-then-delete' as two steps, and no amount

of eventual consistency fixes it. The only correct fix is collapsing check-and-mutate into one atomic, strongly-consistent operation — `DELETE ... RETURNING` in SQL or `GETDEL` /a Lua script in Redis — so the datastore itself guarantees exactly one caller ever sees the content."

Summary — the mental model

Pastebin's core tension is **wildly variable payload size on an otherwise simple CRUD resource**. The fix: **split the object in two** — a tiny, cacheable metadata row (key, owner, TTL, visibility, blob pointer) in a KV/SQL store, and the paste body in object storage behind a CDN, written blob-first so metadata never dangles. Because pastes are immutable, caching is aggressive and invalidation is nearly free — except **burn-after-read**, which bypasses every cache and resolves through one atomic, strongly-consistent conditional delete so exactly one reader ever wins. Cleanup and analytics run asynchronously, off the request path, with object-store lifecycle rules as a backstop. Everything else — key generation, rate limiting, multi-region replication — is standard scaffolding once that core split and that one correctness-critical race are handled.